KUBERNETES — COMPLETE STUDY NOTES

CKA / Interview Ready | YAML + kubectl + Concepts

i. PODS

Pod Manifest Structure

- A Pod is the smallest deployable unit in Kubernetes
- Can contain one or more containers that share network and storage

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: default
  labels:
    app: myapp
spec:
  containers:
  - name: my-container
    image: nginx:latest
    ports:
    - containerPort: 80
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

`apiVersion: v1` → API version for core resources like Pod, Service, ConfigMap

`kind: Pod` → type of Kubernetes resource

`metadata.name` → name of the pod — must be unique within a namespace

`metadata.labels` → key-value pairs used by Services and Selectors to find this pod

`spec.containers` → list of containers to run inside this pod

`resources.requests` → minimum resources the container needs — used for scheduling

`resources.limits` → maximum resources the container is allowed to use

Multi-Container Pods

- Multiple containers inside a single pod share the same network namespace and storage
- They communicate with each other via localhost

```yaml
spec:
  containers:
  - name: main-app
    image: myapp:latest
```

```
  - name: sidecar-logger
    image: fluentd:latest
```

⚠ NOTE: Containers in the same pod share the same IP address and can talk to each other via localhost.

Init Containers

- Run before main containers start
- Must complete successfully before the main container starts
- Used for setup tasks: waiting for a service, loading config, setting permissions

```
spec:
  initContainers:
  - name: init-wait
    image: busybox
    command: ['sh', '-c', 'until nslookup myservice; do echo waiting; sleep 2;
done']
  containers:
  - name: main-app
    image: myapp:latest
```

`initContainers` → runs these containers in order before starting main containers

`command` → overrides the container's default entrypoint command

Static Pods

- Managed directly by the kubelet on a specific node — NOT through the API server
- Defined as YAML files placed in the kubelet's manifest directory
- Default manifest path: /etc/kubernetes/manifests/
- kubelet scans this directory and creates/deletes pods based on files found there
- Control plane components (kube-apiserver, etcd, scheduler) run as static pods

⚠ NOTE: Static pods always have the node name appended to their pod name e.g. kube-apiserver-controlplane

Pod Lifecycle Phases

```
Phase       Description
-------     -----------
Pending     Pod accepted but containers not yet running (scheduling or image pull
in progress)
Running     Pod bound to a node, at least one container is running
Succeeded   All containers completed successfully and will not restart
Failed      All containers terminated, at least one terminated with failure
Unknown     Pod state cannot be determined (usually node communication issue)
```

⚠ NOTE: Pod Status vs Phase: Phase is the overall lifecycle stage. Status is more detailed — includes conditions like Ready, Initialized, ContainersReady.

Container Restart Policy

- **Always** — always restart the container if it stops — default for Deployments
- **OnFailure** — restart only if the container exits with a non-zero exit code
- **Never** — never restart the container regardless of exit code

```yaml
spec:
  restartPolicy: Always   # Always | OnFailure | Never
```

Resource Requests and Limits

- **Requests** — minimum resources guaranteed to the container — used by scheduler to place pod on a node
- **Limits** — maximum resources the container can use — container is throttled or killed if exceeded

```yaml
resources:
  requests:
    memory: "64Mi"    # 64 Mebibytes
    cpu: "250m"       # 250 millicores = 0.25 CPU core
  limits:
    memory: "128Mi"
    cpu: "500m"
```

`250m CPU` → 250 millicores = 0.25 of one CPU core. 1000m = 1 full core.

`64Mi memory` → 64 Mebibytes. Use Mi for Mebibytes, Gi for Gibibytes.

Pod Priority and Preemption

- Higher priority pods can evict lower priority pods from a node when resources are tight
- PriorityClass defines a priority value — higher number = higher priority

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
---
spec:
  priorityClassName: high-priority
```

Probes

- **Liveness Probe** — checks if container is alive — restarts container if it fails
- **Readiness Probe** — checks if container is ready to receive traffic — removes from Service endpoints if it fails
- **Startup Probe** — checks if application has started — disables liveness and readiness until it passes

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10   # wait before first check
  periodSeconds: 5          # how often to check
  failureThreshold: 3       # fail this many times before action
  successThreshold: 1       # succeed this many times to pass
  timeoutSeconds: 2         # timeout per check

readinessProbe:
  tcpSocket:
    port: 3306
```

```
startupProbe:
  exec:
    command: ['cat', '/app/ready']
```

`httpGet` → probe type — makes an HTTP GET request to the specified path and port

`tcpSocket` → probe type — tries to open a TCP connection to the port

`exec` → probe type — runs a command inside the container, success = exit code 0

`initialDelaySeconds` → how long to wait after container starts before first probe

`periodSeconds` → how often to run the probe

`failureThreshold` → how many consecutive failures before taking action

Multi-Container Design Patterns

- **Sidecar** — extends main container functionality. Example: log shipper running alongside app, service mesh proxy
- **Ambassador** — proxy between main container and external world. Example: routing DB connections to correct DB
- **Adapter** — transforms output of main container. Example: converting app log format to standard format

ii. REPLICASETS

Purpose of ReplicaSets

- Ensures a specified number of pod replicas are running at any time
- If a pod dies, ReplicaSet creates a new one to maintain the count
- If too many pods exist, ReplicaSet deletes the extras

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: my-container
        image: nginx:latest
```

`replicas: 3` → maintain exactly 3 running pods at all times

`selector.matchLabels` → ReplicaSet manages pods that match these labels

`template` → blueprint for creating new pods when needed

Relationship with Deployments

- You almost NEVER create a ReplicaSet directly
- Deployments create and manage ReplicaSets automatically
- When you update a Deployment, it creates a NEW ReplicaSet and scales down the OLD one
- This is how rolling updates and rollbacks work

⚠ NOTE: ReplicaSet is the mechanism. Deployment is the abstraction you use. Always use Deployments.

Selector and Labels

- ReplicaSet uses selector to find which pods it owns
- Pods must have matching labels to be managed by the ReplicaSet
- If you create a pod with matching labels manually, the ReplicaSet will count it and may delete it

iii. DEPLOYMENTS

What is a Deployment

- Manages ReplicaSets and provides declarative updates to Pods
- Handles rolling updates, rollbacks, and scaling

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: my-container
        image: nginx:1.21
```

Rolling Update

- Default update strategy — updates pods gradually with zero downtime
- Creates new pods before deleting old ones (controlled by maxSurge and maxUnavailable)

```
kubectl set image deployment/my-deployment my-container=nginx:1.22
kubectl rollout status deployment/my-deployment
```

```
kubectl rollout history deployment/my-deployment
```

`maxSurge: 1` → how many extra pods above desired count can exist during update

`maxUnavailable: 1` → how many pods can be unavailable during update

Rollback

- Revert to a previous version if something goes wrong

```
kubectl rollout undo deployment/my-deployment              # rollback to previous
kubectl rollout undo deployment/my-deployment --to-revision=2  # rollback to
specific revision
kubectl rollout history deployment/my-deployment           # see revision history
```

Scaling

```
kubectl scale deployment/my-deployment --replicas=5
kubectl autoscale deployment/my-deployment --min=2 --max=10 --cpu-percent=80
```

Deployment Strategies

- **RollingUpdate** — gradual replacement of old pods with new ones — zero downtime
- **Recreate** — terminate ALL old pods then create ALL new ones — causes downtime but clean cutover
- **Canary Deployment** — deploy new version to small subset of users first, then gradually increase traffic
- **Blue/Green Deployment** — run old (blue) and new (green) versions simultaneously, switch traffic all at once
- **A/B Testing** — route different users to different versions based on criteria (headers, cookies, location)
- **Shadow Deployment** — send copy of real traffic to new version without affecting users — test in production

```
# Recreate strategy
strategy:
  type: Recreate

# RollingUpdate strategy
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 25%
    maxUnavailable: 25%
```

iv. DAEMONSETS

What is a DaemonSet

- Ensures ONE copy of a pod runs on EVERY node in the cluster
- When a new node is added, DaemonSet automatically adds a pod to it
- When a node is removed, the pod is garbage collected

```
apiVersion: apps/v1
kind: DaemonSet
```

```yaml
metadata:
  name: my-daemonset
spec:
  selector:
    matchLabels:
      app: monitoring-agent
  template:
    metadata:
      labels:
        app: monitoring-agent
    spec:
      containers:
      - name: agent
        image: datadog/agent:latest
```

Use Cases

- Log collectors (Fluentd, Filebeat) — run on every node to collect logs
- Monitoring agents (Datadog, Prometheus Node Exporter) — collect metrics from every node
- Network plugins (Calico, Flannel) — must run on every node
- Storage daemons — run on every node that needs access to storage

Relationship with Taints and Tolerations

- By default DaemonSet pods do NOT run on tainted nodes (e.g. control plane nodes)
- To run a DaemonSet on tainted nodes, add tolerations to the pod spec

```yaml
spec:
  template:
    spec:
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        effect: NoSchedule
        operator: Exists
```

v. NAMESPACES

Purpose

- Logical separation of resources within a cluster

⚠ NOTE: Namespaces provide logical isolation — they do NOT provide network isolation by default. Pods in different namespaces can still communicate unless Network Policies restrict it.

Default Namespaces

- **default** — where resources go if no namespace is specified
- **kube-system** — Kubernetes system components — apiserver, etcd, scheduler, coredns
- **kube-public** — publicly readable by all users, used for cluster info

- **kube-node-lease** — holds node heartbeat Lease objects — used for node health detection

Namespace Commands

```
kubectl get namespaces
kubectl create namespace dev
kubectl get pods -n kube-system                 # get pods in specific namespace
kubectl get pods --all-namespaces               # get pods across all namespaces
kubectl get pods -A                             # shorthand for --all-namespaces
kubectl config set-context --current --namespace=dev  # set default namespace
```

Resource Quota per Namespace

- Limits the total resources (CPU, memory, number of pods) a namespace can consume

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 4Gi
    limits.cpu: "8"
    limits.memory: 8Gi
```

Limit Range per Namespace

- Sets default and maximum resource requests/limits for containers in a namespace
- Applies to containers that do not specify their own requests/limits

```
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-limitrange
  namespace: dev
spec:
  limits:
  - default:
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:
      cpu: "250m"
      memory: "128Mi"
    type: Container
```

vi. SERVICES

What is a Service

- Provides a stable IP and DNS name to access a set of pods
- Pods are ephemeral — their IPs change. Services provide a stable endpoint.

- Uses label selectors to find which pods to route traffic to

Service Types

- **ClusterIP** — default. Exposes service on internal cluster IP only — not accessible from outside
- **NodePort** — exposes service on each node's IP at a static port (30000-32767) — accessible from outside
- **LoadBalancer** — provisions an external load balancer (cloud provider) — accessible from internet
- **ExternalName** — maps service to an external DNS name — no selector, no proxying
- **Headless Service** — clusterIP: None — returns pod IPs directly, used for StatefulSets and service discovery

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP         # ClusterIP | NodePort | LoadBalancer | ExternalName
  selector:
    app: myapp             # routes traffic to pods with this label
  ports:
  - protocol: TCP
    port: 80               # port the service listens on
    targetPort: 8080       # port on the pod to forward traffic to
    nodePort: 30080        # only for NodePort type (30000-32767)
```

`selector.app: myapp` → service routes traffic to all pods with label app=myapp

`port: 80` → the port clients use to connect to the service

`targetPort: 8080` → the port on the actual pod container that receives the traffic

`nodePort: 30080` → external port on each node — only needed for NodePort type

Endpoints

- Kubernetes automatically creates an Endpoints object for each service
- Endpoints list the IP:port of all pods that match the service selector

```
kubectl get endpoints my-service
```

Service Discovery

- Every service gets a DNS name automatically
- Format within same namespace: my-service
- Format across namespaces: my-service.namespace.svc.cluster.local

```
# From a pod in same namespace:
curl http://my-service:80

# From a pod in different namespace:
curl http://my-service.dev.svc.cluster.local:80
```

Selector-based vs Manual Endpoints

- **Selector-based** — service uses selector — Kubernetes auto-manages Endpoints

- **Manual Endpoints** — service has no selector — you manually create Endpoints. Use case: routing to external database

vii. CONFIGMAPS

What is a ConfigMap

- Stores non-sensitive configuration data as key-value pairs
- Decouples configuration from container images
- Changes to ConfigMap can be reflected in pods without rebuilding images

Creation Methods

```
# --from-literal
kubectl create configmap my-config --from-literal=DB_HOST=localhost --from-
literal=DB_PORT=5432

# --from-file (single file)
kubectl create configmap my-config --from-file=app.properties

# --from-file (directory)
kubectl create configmap my-config --from-file=./config-dir/

# --from-env-file
kubectl create configmap my-config --from-env-file=.env

# From manifest YAML
kubectl apply -f configmap.yaml
```

ConfigMap YAML Manifest

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  DB_HOST: "localhost"          # simple key-value pair
  DB_PORT: "5432"
  app.properties: |             # multi-line block literal
    server.port=8080
    server.timeout=30
    logging.level=INFO
```

`data:` → holds the key-value pairs

`|` → block literal — preserves newlines, used for multi-line config files

Mounting ConfigMap to Pods

As Environment Variables (individual keys)

```
env:
- name: DATABASE_HOST
  valueFrom:
```

```
    configMapKeyRef:
      name: my-config
      key: DB_HOST
```

`configMapKeyRef` → reference a specific key from a ConfigMap

As Environment Variables (all keys at once)

```
envFrom:
- configMapRef:
    name: my-config
```

`envFrom` → load ALL keys from the ConfigMap as environment variables at once

As Volume Mount

```
spec:
  volumes:
  - name: config-volume
    configMap:
      name: my-config
  containers:
  - name: my-container
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
```

`mountPath: /etc/config` → each key in the ConfigMap becomes a file inside /etc/config/

`configMap.name` → which ConfigMap to mount as files

Immutable ConfigMap

- Once set to immutable, the ConfigMap cannot be updated
- Better performance — kubelet stops watching for changes

```
immutable: true
```

Update Propagation Delay

- When a ConfigMap is updated and mounted as a volume — pods see changes after a delay (up to 1-2 minutes)
- When a ConfigMap is used as environment variable — pods do NOT see changes until restarted

⚠ NOTE: Environment variable-based ConfigMap changes require pod restart. Volume-mounted ConfigMaps update automatically after propagation delay.

viii. SECRETS

What is a Secret

- Stores sensitive data like passwords, tokens, certificates
- Values are Base64 encoded by default — NOT encrypted

⚠ NOTE: Secrets are NOT encrypted in etcd by default. They are only Base64 encoded. Enable encryption at rest for production security.

Secret Types

- **Generic (Opaque)** — default type — arbitrary key-value data
- **kubernetes.io/tls** — TLS certificate and key — used with Ingress
- **kubernetes.io/dockerconfigjson** — Docker registry credentials — used to pull private images

Creation Methods

```
# --from-literal
kubectl create secret generic my-secret --from-literal=password=mysecret123

# --from-file
kubectl create secret generic my-secret --from-file=ssh-private-key=~/.ssh/id_rsa

# TLS secret
kubectl create secret tls my-tls --cert=tls.crt --key=tls.key

# Docker registry secret
kubectl create secret docker-registry my-registry \
  --docker-server=registry.example.com \
  --docker-username=user \
  --docker-password=pass
```

Secret YAML Manifest

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  password: bXlzZWNyZXQxMjM=   # base64 encoded value
  username: YWRtaW4=
```

`bXlzZWNyZXQxMjM=` → base64 encoding of "mysecret123" — run: echo -n "mysecret123" | base64

Mounting Secrets to Pods

```
# As environment variable
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: my-secret
      key: password

# As volume mount
volumes:
- name: secret-volume
  secret:
    secretName: my-secret
volumeMounts:
- name: secret-volume
  mountPath: /etc/secrets
```

```
  readOnly: true
```

Base64 Encoding/Decoding

```
echo -n "mysecret" | base64           # encode
echo "bXlzZWNyZXQ=" | base64 --decode # decode
```

Security Best Practices

- Secrets not encrypted by default in etcd — only Base64 encoded
- Enable encryption at rest using EncryptionConfiguration
- Use RBAC to restrict which service accounts can access secrets
- Use immutable: true to prevent accidental modification
- Consider external secret managers (Azure Key Vault, HashiCorp Vault) for production

ix. STORAGE / VOLUMES

Volume vs VolumeMount Distinction

- **Volume** — defined at the pod spec level — declares what storage exists
- **VolumeMount** — defined at the container level — declares where to mount the volume inside the container

⚠ NOTE: Volume = WHAT storage. VolumeMount = WHERE to attach it inside the container. Both are required.

Volume Types

emptyDir

- Created when pod is assigned to a node, deleted when pod is removed
- Shared between all containers in the same pod
- Disk-based (default) or Memory-based (medium: Memory)

```
volumes:
- name: cache-vol
  emptyDir: {}              # disk-based

- name: mem-vol
  emptyDir:
    medium: Memory          # memory-based (uses RAM, faster)
    sizeLimit: 256Mi
```

hostPath

- Mounts a file or directory from the HOST NODE's filesystem into the pod
- Persists after pod deletion — data stays on the node
- Risk: pod is tied to a specific node

```
volumes:
- name: host-vol
  hostPath:
    path: /data/myapp
```

```
    type: DirectoryOrCreate  # create directory if not exists
```

Persistent Volume (PV)

- A piece of storage in the cluster provisioned by an administrator
- Exists independently of any pod — lifecycle is separate from pods

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: standard
  hostPath:
    path: /data/pv
```

Access Modes

```
ReadWriteOnce (RWO)   # read/write by ONE node at a time
ReadOnlyMany  (ROX)   # read-only by MANY nodes simultaneously
ReadWriteMany (RWX)   # read/write by MANY nodes simultaneously
```

⚠ NOTE: Most block storage (Azure Disk, AWS EBS) only supports ReadWriteOnce. NFS supports ReadWriteMany.

Reclaim Policies

- **Retain** — PV is kept after PVC is deleted — admin must manually reclaim. Data is safe.
- **Delete** — PV and underlying storage are deleted when PVC is deleted
- **Recycle** — deprecated — basic scrub (rm -rf) and make available again

PV Status

```
Available  # PV exists and has not been bound to a PVC yet
Bound      # PV is bound to a PVC
Released   # PVC was deleted but PV has not been reclaimed yet
Failed     # automatic reclamation failed
```

Persistent Volume Claim (PVC)

- A request for storage by a user/pod
- Pod references PVC — PVC binds to a matching PV

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
```

```
  storageClassName: standard
```

`storageClassName` → must match the storageClassName of the PV for binding

`accessModes` → must be compatible with the PV's access modes

Binding to Specific PV using volumeName

```
spec:
  volumeName: my-specific-pv   # bind to this exact PV by name
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

Using PVC in a Pod

```
volumes:
- name: my-storage
  persistentVolumeClaim:
    claimName: my-pvc
containers:
- name: my-container
  volumeMounts:
  - name: my-storage
    mountPath: /data
```

StorageClass

- Defines HOW storage is provisioned — the "class" of storage
- **Static Provisioning** — admin manually creates PVs. PVC then binds to existing PV.
- **Dynamic Provisioning** — StorageClass automatically creates a PV when a PVC is created
- **Local Path Provisioner** — provisions storage from local disk on the node

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: kubernetes.io/azure-disk
parameters:
  skuName: Premium_LRS
reclaimPolicy: Delete
allowVolumeExpansion: true
```

`provisioner` → which plugin to use to create the storage (Azure, AWS, GCP etc.)

`reclaimPolicy` → what happens to the PV when the PVC is deleted

x. SCHEDULING

kube-scheduler Role

- Watches for newly created pods with no assigned node
- Selects the best node for each pod based on: resource availability, taints/tolerations, affinity rules, node selectors

Node Selector

- Simplest way to constrain pods to specific nodes
- Pod will only be scheduled on nodes with matching labels

```
# Label a node first
kubectl label node node1 disktype=ssd

# Then use nodeSelector in pod spec
spec:
  nodeSelector:
    disktype: ssd
```

Node Affinity

- More expressive than nodeSelector — supports operators like In, NotIn, Exists, Gt, Lt
- **requiredDuringSchedulingIgnoredDuringExecution** — HARD rule — pod MUST be scheduled on matching node. If no match, pod stays Pending.
- **preferredDuringSchedulingIgnoredDuringExecution** — SOFT rule — scheduler prefers matching node but will schedule elsewhere if no match.

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
            - nvme
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: zone
            operator: In
            values:
            - us-east-1a
```

`IgnoredDuringExecution` → if node labels change AFTER pod is scheduled, pod is NOT evicted

`operator: In` → node label value must be one of the listed values

`weight: 1` → weight of this preference (1-100) — higher weight = stronger preference

Taints and Tolerations

- Taints are applied to NODES — they repel pods from being scheduled on that node
- Tolerations are applied to PODS — they allow pods to be scheduled on tainted nodes

```
# Apply a taint to a node
kubectl taint nodes node1 key=value:NoSchedule

# Remove a taint from a node
kubectl taint nodes node1 key=value:NoSchedule-
```

```yaml
# Add toleration to a pod
spec:
  tolerations:
  - key: "key"
    operator: "Equal"
    value: "value"
    effect: "NoSchedule"
```

Taint Effects

- **NoSchedule** — new pods without matching toleration will NOT be scheduled on this node
- **PreferNoSchedule** — scheduler tries to avoid placing pods here but will if no other option
- **NoExecute** — new pods won't be scheduled AND existing pods without toleration will be EVICTED

Toleration Operators

- **Equal** — key and value must match the taint exactly
- **Exists** — only key must match — value is ignored. Used when taint value is unknown.

Taint Based Evictions and Toleration Seconds

```yaml
tolerations:
- key: "node.kubernetes.io/not-ready"
  operator: "Exists"
  effect: "NoExecute"
  tolerationSeconds: 300    # stay on node for 300 seconds before eviction
```

`tolerationSeconds: 300` → pod tolerates the taint for 300 seconds before being evicted — default is 300s

Scheduling Pods without a Scheduler

- Manually assign a pod to a node using nodeName field

```yaml
spec:
  nodeName: node1   # bypass scheduler — directly assign to this node
```

⚠ NOTE: nodeName bypasses the scheduler entirely. Pod is placed directly on the specified node.

Pod Disruption Budget (PDB)

- Limits the number of pods that can be down simultaneously during voluntary disruptions
- Voluntary disruptions: node draining, rolling updates, cluster upgrades

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-pdb
spec:
  minAvailable: 2       # at least 2 pods must always be available
  # maxUnavailable: 1   # OR at most 1 pod can be unavailable
  selector:
    matchLabels:
      app: myapp
```

xi. RESOURCE MANAGEMENT

Requests vs Limits

- **Requests** — what the container is GUARANTEED — used by scheduler to find a node with enough capacity
- **Limits** — the MAXIMUM the container can use — container is CPU-throttled or OOM-killed if exceeded

```
resources:
  requests:
    cpu: "250m"      # 250 millicores
    memory: "128Mi"  # 128 Mebibytes
  limits:
    cpu: "500m"
    memory: "256Mi"
```

Capacity vs Allocatable vs Allocated

- **Capacity** — total hardware resources on the node
- **Allocatable** — capacity minus what Kubernetes system components reserve — what's available for pods
- **Allocated** — sum of all pod requests currently scheduled on the node

```
kubectl describe node node1   # shows Capacity, Allocatable, and Allocated
Resources
```

Horizontal Pod Autoscaler (HPA)

- Automatically scales the number of pod replicas based on metrics
- Metrics Server must be installed for HPA to work

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
    scaleUp:
      stabilizationWindowSeconds: 0
```

`minReplicas / maxReplicas` → the autoscaler will scale between these bounds
`averageUtilization: 80` → scale up when average CPU usage across pods exceeds 80%
`stabilizationWindowSeconds` → how long to wait before scaling down to avoid flapping

```
kubectl autoscale deployment my-deployment --cpu-percent=80 --min=2 --max=10
```

Vertical Pod Autoscaler (VPA)

- Automatically adjusts CPU and memory requests/limits for containers
- Does NOT change number of replicas — changes resource sizes instead

Recommendation Modes

- **Off** — only provides recommendations — does not apply any changes
- **Initial** — sets resources only when pod is created — does not change running pods
- **Auto** — continuously updates resources — may restart pods to apply changes

VPA Components

- **Recommender** — monitors usage and provides resource recommendations
- **Updater** — evicts pods that need to be updated with new resource values
- **Admission Controller** — sets resource requests/limits on new pods based on recommendations

xii. NETWORK POLICIES

Default Behavior

⚠ NOTE: By default ALL traffic is allowed between all pods in a Kubernetes cluster. Network Policies are used to RESTRICT traffic. A network plugin that supports Network Policies (like Calico) must be installed — Flannel does NOT support it.

Policy Types

- **Ingress** — controls incoming traffic TO the selected pods
- **Egress** — controls outgoing traffic FROM the selected pods

Network Policy YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: my-network-policy
  namespace: default
spec:
  podSelector:                  # select which pods this policy applies to
    matchLabels:
      app: myapp
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
```

```yaml
    - podSelector:              # allow from pods with this label
        matchLabels:
          role: frontend
    - namespaceSelector:        # allow from pods in this namespace
        matchLabels:
          name: monitoring
    - ipBlock:                  # allow from this IP range
        cidr: 192.168.0.0/24
        except:
        - 192.168.0.5/32        # except this specific IP
    ports:
    - protocol: TCP
      port: 8080
      endPort: 8090             # port range
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - port: 5432
```

`podSelector: {}` → empty selector = select ALL pods in the namespace

`from` → defines sources for ingress (incoming) traffic

`to` → defines destinations for egress (outgoing) traffic

`except` → exclude these IPs from the allowed IP block

`endPort` → creates a port range — allow traffic on ports 8080 through 8090

xiii. INGRESS

What is Ingress

- Manages external HTTP/HTTPS access to services inside the cluster
- Provides host-based and path-based routing from a single external IP
- Ingress Controller (like nginx) must be deployed — it reads Ingress rules and configures routing

⚠ NOTE: Ingress resource = the routing rules. Ingress Controller = the actual component (nginx pod) that enforces those rules. You need BOTH.

Ingress YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: myapp.example.com
    http:
```

```
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 8080
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
  defaultBackend:
    service:
      name: default-service
      port:
        number: 80
```

`host: myapp.example.com` → host-based routing — match on hostname

`path: /api` → path-based routing — route /api/* to api-service

`pathType: Prefix` → match any path starting with /api

`ingressClassName: nginx` → which Ingress Controller handles this resource

`defaultBackend` → fallback service when no rules match

Ingress with TLS

```
# Step 1: Create self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt -subj "/CN=myapp.example.com"

# Step 2: Create TLS secret
kubectl create secret tls my-tls-secret --cert=tls.crt --key=tls.key

# Step 3: Reference in Ingress
spec:
  tls:
  - hosts:
    - myapp.example.com
    secretName: my-tls-secret   # TLS terminates at Ingress Controller
  rules:
  - host: myapp.example.com
```

`tls.secretName` → Ingress Controller reads the cert/key from this secret for HTTPS termination

xiv. GATEWAY API

Difference from Ingress

- Gateway API is the newer, more expressive successor to Ingress
- Ingress is limited — all routing in one resource, hard to extend

- Gateway API separates concerns into multiple resources: GatewayClass, Gateway, HTTPRoute
- Supports more routing features: header-based routing, traffic splitting, redirects

Key Resources

- **GatewayClass** — defines which controller handles this gateway (like IngressClass)
- **Gateway** — defines a load balancer — listens on ports, terminates TLS
- **HTTPRoute** — defines HTTP routing rules — attaches to a Gateway

```yaml
# Gateway resource
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    port: 80
    protocol: HTTP

# HTTPRoute resource
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-route
spec:
  parentRefs:
  - name: my-gateway
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /api
    backendRefs:
    - name: api-service
      port: 8080
```

`parentRefs` → which Gateway this HTTPRoute attaches to

Ingress vs Ingress Controller vs Gateway API

```
Ingress            = routing rules resource (what you write)
Ingress Controller = nginx pod that reads Ingress rules and routes traffic
Gateway API        = newer alternative — splits into Gateway + HTTPRoute resources
                     more powerful, more flexible, better role separation
```

xv. HELM

What is Helm

- Package manager for Kubernetes — like apt/yum but for K8s applications
- Packages Kubernetes manifests into Charts — reusable, versioned bundles

- Makes deploying complex applications simple with a single command

Helm Chart Structure

```
my-chart/
├── Chart.yaml          # chart metadata (name, version, description)
├── values.yaml         # default configuration values
├── templates/          # Kubernetes manifest templates
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
└── charts/             # dependent charts
```

Helm Repository Commands

```
helm repo add bitnami https://charts.bitnami.com/bitnami   # add a repo
helm repo update                                            # refresh repo index
helm search repo nginx                                      # search within added
repos
helm search hub nginx                                       # search Artifact Hub
(public)
```

Helm Commands

```
helm install my-release bitnami/nginx          # install a chart as a release
helm install my-release ./my-chart             # install from local chart
directory
helm upgrade my-release bitnami/nginx          # upgrade an existing release
helm rollback my-release 1                     # rollback to revision 1
helm uninstall my-release                      # delete a release
helm list                                      # list all releases
helm list -n dev                               # list releases in namespace
helm template my-release ./my-chart            # render templates without
installing
helm show values bitnami/nginx                 # show default values for a chart
helm get values my-release                     # show values used for installed
release
```

Values Override

```
# Override using --set (single value)
helm install my-release bitnami/nginx --set replicaCount=3

# Override using --values (-f) with a file (preferred for multiple values)
helm install my-release bitnami/nginx -f my-values.yaml

# my-values.yaml
replicaCount: 3
image:
  tag: "1.21"
service:
  type: LoadBalancer
```

`--set` → quick override for single values — good for CLI one-offs

`--values (-f)` → override with a file — preferred for multiple values, version-controllable

Skipping CRDs

```
helm install my-release my-chart --skip-crds
helm install my-release my-chart --set crds.install=false
```

- Use when CRDs are already installed — prevents conflicts with existing CRD versions

Namespaces in Helm

```
helm install my-release bitnami/nginx --namespace dev --create-namespace
```

`--namespace dev` → install into the dev namespace

`--create-namespace` → create the namespace if it does not exist

xvi. RBAC

What is RBAC

- Role-Based Access Control — controls WHO can do WHAT on which resources
- 4 key resources: Role, ClusterRole, RoleBinding, ClusterRoleBinding

Role vs ClusterRole

- **Role** — namespace-scoped — grants permissions within a specific namespace
- **ClusterRole** — cluster-wide — grants permissions across all namespaces or for cluster-scoped resources

```
# Role — namespace-scoped
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]             # "" means core API group
  resources: ["pods"]
  verbs: ["get", "list", "watch"]

# ClusterRole — cluster-wide
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list"]
```

`apiGroups: [""]` → empty string = core API group (pods, services, configmaps, secrets)

`apiGroups: ["apps"]` → apps API group (deployments, replicasets, daemonsets)

RoleBinding and ClusterRoleBinding

```
# RoleBinding — binds a Role to a user/serviceaccount in a namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
```

```yaml
  namespace: dev
subjects:
- kind: User
  name: allen
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

`subjects` → who gets the permissions — User, Group, or ServiceAccount

`roleRef` → which Role or ClusterRole to bind

⚠ NOTE: RoleBinding can also bind a ClusterRole but only within the specified namespace.

Service Accounts

- Identity for processes running inside pods — NOT for human users
- Every namespace has a default service account
- Pods use service accounts to authenticate with the Kubernetes API

```yaml
# Create a service account
kubectl create serviceaccount my-sa -n dev

# Use in pod spec
spec:
  serviceAccountName: my-sa
  automountServiceAccountToken: false   # disable auto token mounting
```

Verbs and API Groups

```
Verbs:      get | list | watch | create | update | patch | delete |
deletecollection

API Groups:
""            → core group: pods, services, configmaps, secrets, namespaces, nodes
"apps"        → deployments, replicasets, daemonsets, statefulsets
"batch"       → jobs, cronjobs
"networking.k8s.io" → ingresses, networkpolicies
"rbac.authorization.k8s.io" → roles, rolebindings, clusterroles
```

xvii. SECURITY CONTEXT

What is Security Context

- Defines privilege and access control settings for a pod or container
- Pod-level settings apply to all containers in the pod
- Container-level settings override pod-level settings

Security Context YAML

```yaml
spec:
  securityContext:           # pod-level
```

```
    runAsUser: 1000          # run all containers as user ID 1000
    runAsGroup: 3000         # primary group ID 3000
    fsGroup: 2000            # group ownership of volumes
    runAsNonRoot: true       # prevent running as root user
  containers:
  - name: my-container
    securityContext:         # container-level (overrides pod-level)
      allowPrivilegeEscalation: false  # no privilege escalation
      readOnlyRootFilesystem: true     # root filesystem is read-only
      capabilities:
        add: ["NET_ADMIN"]   # add specific Linux capabilities
        drop: ["ALL"]        # drop all capabilities first
```

`runAsUser: 1000` → container runs as UID 1000 — not root

`runAsGroup: 3000` → primary GID for the container process

`fsGroup: 2000` → volumes mounted by this pod will be owned by GID 2000

`runAsNonRoot: true` → Kubernetes rejects container if it tries to run as root

`allowPrivilegeEscalation: false` → prevents process from gaining more privileges than its parent

`readOnlyRootFilesystem: true` → container cannot write to its own filesystem — security hardening

`capabilities.drop: ["ALL"]` → drop ALL Linux capabilities — best practice, then add back only what is needed

`capabilities.add: ["NET_ADMIN"]` → add back only the specific capability needed

xviii. CLUSTER ARCHITECTURE

Control Plane Components

- **kube-apiserver** — the front door of the cluster — all kubectl commands go through it. Validates and processes API requests.
- **etcd** — distributed key-value store — stores ALL cluster state. Only the apiserver talks directly to etcd.
- **kube-scheduler** — watches for unscheduled pods and assigns them to nodes based on resource availability and constraints
- **kube-controller-manager** — runs all controller loops — monitors cluster state and makes changes to reach desired state
- **Cloud Controller Manager** — interacts with cloud provider APIs (Azure, AWS) — manages load balancers, storage, nodes

Controllers inside kube-controller-manager

- Node Controller — monitors node health, marks NotReady after 40s grace period, evicts pods after 5 min
- Replication Controller — ensures correct number of pod replicas
- Deployment Controller — manages Deployment rollouts and ReplicaSets
- Service Account Controller — creates default service accounts in new namespaces
- Namespace Controller — handles namespace creation and deletion

Worker Node Components

- **kubelet** — agent on each node — communicates with apiserver, ensures containers in pods are running and healthy
- **kube-proxy** — network proxy on each node — maintains iptables rules for Service routing
- **Container Runtime** — software that runs containers — containerd, CRI-O, or cri-dockerd

Container Runtime Interface (CRI)

- Standard API between kubelet and container runtimes
- **containerd** — most common — lightweight, industry standard
- **CRI-O** — lightweight runtime built specifically for Kubernetes
- **cri-dockerd** — Docker adapter — allows using Docker Engine via CRI (Docker itself is not CRI-compliant)

crictl Tool

- CLI tool for interacting with CRI-compatible runtimes directly (like docker CLI but for CRI)

```
crictl ps                        # list running containers
crictl images                    # list images
crictl logs <container-id>       # view container logs
crictl pods                      # list pods
crictl exec -it <id> /bin/sh     # exec into container
```

Cluster Architecture Flow

```
kubectl apply -f pod.yaml
       ↓
kube-apiserver   (validates request, stores in etcd)
       ↓
etcd             (stores desired state)
       ↓
kube-scheduler   (watches for unscheduled pod, assigns node)
       ↓
kubelet on node  (sees pod assigned to its node, tells container runtime)
       ↓
container runtime (pulls image, starts container)
       ↓
kubelet          (reports status back to apiserver)
```

xix. OBSERVABILITY AND LOGGING

kubectl logs

```
kubectl logs my-pod                          # logs from single-container pod
kubectl logs my-pod -c my-container          # logs from specific container (-c
flag)
kubectl logs my-pod --previous               # logs from previous crashed
container
kubectl logs my-pod -f                       # follow/stream live logs
kubectl logs my-pod --tail=50                # last 50 lines
kubectl logs my-pod --since=1h               # logs from last 1 hour
```

⚠ **NOTE: For multi-container pods, always use -c <container-name> to specify which container's logs you want.**

Kubernetes Events

- Events record what happened in the cluster — very useful for debugging
- Default retention: 1 hour — events are deleted after 1 hour
- Events are namespace-scoped

```
kubectl get events -n default
kubectl get events --field-selector reason=BackOff
kubectl get events --field-selector involvedObject.name=my-pod
kubectl describe pod my-pod   # shows events at the bottom — most common debug
method
```

Field Selectors

```
kubectl get pods --field-selector metadata.namespace=dev
kubectl get pods --field-selector metadata.name=my-pod
kubectl get events --field-selector involvedObject.name=my-pod
kubectl get pods --field-selector status.phase=Running
```

Cluster Component Logs

```
# For systemd-managed components (kubeadm clusters)
journalctl -u kubelet                          # kubelet logs
journalctl -u kubelet --since "1 hour ago"     # last 1 hour
journalctl -u kubelet --until "2024-01-01"     # until a date
journalctl -u kubelet -f                       # follow live

# For static pod components (in /var/log/pods/ or via kubectl)
kubectl logs -n kube-system kube-apiserver-controlplane
kubectl logs -n kube-system etcd-controlplane
```

Docker Logging Drivers

- **json-file (default)** — logs stored as JSON files on the host — kubectl logs reads these
- **none** — disables logging entirely
- **journald** — logs go to systemd journal
- **syslog** — logs forwarded to syslog
- **fluentd** — logs forwarded to Fluentd aggregator
- **awslogs / splunk** — logs forwarded to cloud logging services

Metrics Server and kubectl top

- Metrics Server collects CPU and memory usage from kubelets
- Required for HPA and kubectl top to work

```
kubectl top nodes           # CPU and memory usage per node
kubectl top pods            # CPU and memory usage per pod
kubectl top pods -n dev     # in specific namespace
kubectl top pods --containers  # breakdown by container
```

xx. PROBES — DETAILED

Probe Types

- **Liveness Probe** — Is the container alive? If fails → container is restarted
- **Readiness Probe** — Is the container ready to serve traffic? If fails → removed from Service endpoints
- **Startup Probe** — Has the application finished starting? Disables liveness/readiness until it passes. For slow-starting apps.

Probe Mechanisms

```
# HTTP GET probe
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
    httpHeaders:
    - name: X-Custom-Header
      value: Awesome

# TCP Socket probe
readinessProbe:
  tcpSocket:
    port: 3306

# Exec (command) probe
startupProbe:
  exec:
    command:
    - cat
    - /app/ready
```

`httpGet` → success = HTTP response 200-399. Failure = anything else.

`tcpSocket` → success = TCP connection opens. Failure = connection refused.

`exec` → success = exit code 0. Failure = non-zero exit code.

Probe Configuration Parameters

```
initialDelaySeconds: 10   # wait 10s after container starts before first probe
periodSeconds: 5          # run probe every 5 seconds
failureThreshold: 3       # fail 3 times in a row before taking action
successThreshold: 1       # succeed 1 time to be considered healthy again
timeoutSeconds: 2         # each probe attempt times out after 2 seconds
```

⚠ NOTE: Liveness and Readiness probes run throughout the container lifecycle. Startup probe only runs once at startup and then stops.

xxi. EDITING AND QUERYING RESOURCES

kubectl edit vs patch vs apply

```
kubectl edit deployment my-deployment      # open in editor, save to apply
```

```
kubectl patch deployment my-deployment -p '{"spec":{"replicas":5}}'  # patch
inline
kubectl apply -f deployment.yaml           # declarative — apply from file
```

- **kubectl create** — imperative — creates resource. Fails if resource already exists.
- **kubectl apply** — declarative — creates OR updates resource. Safe to run multiple times.

⚠ NOTE: Always prefer kubectl apply over kubectl create for production use. apply is idempotent — safe to run multiple times.

Mutable vs Immutable Fields

- Some fields cannot be changed on a running resource — you must delete and recreate
- Example immutable fields: pod spec containers image (in some cases), pod spec nodeName

```
# Check if field is mutable
kubectl explain pod.spec.containers.image

# For immutable changes, delete and recreate
kubectl replace --force -f pod.yaml
```

JSONPath

- Query specific fields from kubectl output

```
# Root object $
kubectl get pods -o jsonpath="{.items[0].metadata.name}"

# Wildcards *
kubectl get pods -o jsonpath="{.items[*].metadata.name}"

# Array indexing
kubectl get pods -o jsonpath="{.items[0].spec.containers[0].image}"

# range and end — iterate over list
kubectl get pods -o jsonpath="{range .items[*]}{.metadata.name}{'\n'}{end}"

# Custom columns output
kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase
```

`.items[*]` → wildcard — select all items in the list

`{range .items[*]}...{end}` → iterate over each item in the list

`custom-columns` → define your own table columns with custom headers and JSONPath values

kubectl dry-run

```
# Client dry-run — validate locally without sending to server
kubectl apply -f pod.yaml --dry-run=client

# Server dry-run — send to server for validation without persisting
kubectl apply -f pod.yaml --dry-run=server

# Generate YAML quickly using dry-run
kubectl run my-pod --image=nginx --dry-run=client -o yaml > pod.yaml
kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml > dep.yaml
```

⚠ **NOTE: dry-run=client is fast but less thorough. dry-run=server validates against admission controllers.**

kubectl diff and kubectl describe

```
kubectl diff -f deployment.yaml      # show diff between local file and live
cluster state
kubectl describe pod my-pod          # detailed info + events — first step in
debugging
kubectl describe node node1          # node details, capacity, conditions, pods on
it
```

xxii. CLUSTER MANAGEMENT

Draining Nodes

- **kubectl drain** — safely evicts all pods from a node and marks it unschedulable — for maintenance

```
kubectl drain node1 --ignore-daemonsets --delete-emptydir-data


# --ignore-daemonsets  → ignore DaemonSet pods (they cannot be evicted)
# --delete-emptydir-data → delete pods with emptyDir volumes
# --force              → evict even pods not managed by a controller
```

⚠ **NOTE: drain = cordon + evict all pods. After drain, node is both unschedulable AND existing pods are gone.**

Cordon and Uncordon

```
kubectl cordon node1     # mark node as unschedulable — no new pods will be
scheduled
kubectl uncordon node1   # mark node as schedulable again
```

- cordon ONLY prevents new pods from being scheduled — existing pods keep running
- drain = cordon + evicts all pods

Node Maintenance Workflow

```
1. kubectl drain node1 --ignore-daemonsets     # evict pods, mark unschedulable
2. # perform maintenance on node (OS update, hardware etc.)
3. kubectl uncordon node1                      # mark schedulable again
```

Version Skew Policy

- Kubernetes version format: major.minor.patch (e.g. 1.28.3)

```
Component version rules:
kube-apiserver         → must be upgraded FIRST, must be newest
kubelet                → can be up to 2 minor versions BEHIND apiserver
kube-proxy             → can be up to 2 minor versions behind apiserver
kubectl                → can be 1 minor version ahead or behind apiserver
controller-manager     → must be same or 1 minor version behind apiserver
scheduler              → must be same or 1 minor version behind apiserver
```

⚠ **NOTE: Always upgrade the control plane (apiserver) first before upgrading worker nodes.**

kubeadm Cluster Setup

Master Node Setup

```
# Initialize the cluster on master node
kubeadm init --pod-network-cidr=10.244.0.0/16

# Configure kubectl
mkdir -p $HOME/.kube
cp /etc/kubernetes/admin.conf $HOME/.kube/config

# Install a network plugin (Calico or Flannel)
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

Worker Node Join

```
# On master: get join token
kubeadm token create --print-join-command

# On worker node: join the cluster
kubeadm join <master-ip>:6443 --token <token> --discovery-token-ca-cert-hash
sha256:<hash>
```

xxiii. TROUBLESHOOTING

Pod Troubleshooting

Common Pod Error States

- **CrashLoopBackOff** — container keeps crashing and restarting. Check logs: kubectl logs my-pod --previous
- **ImagePullBackOff / ErrImagePull** — cannot pull the container image. Check image name, tag, registry credentials.
- **OOMKilled** — container exceeded memory limit and was killed. Increase memory limit.
- **Pending** — pod cannot be scheduled. Check: insufficient resources, node selector mismatch, taint/toleration issue

```
# Step 1: Check pod status
kubectl get pods

# Step 2: Describe pod — check Events section at bottom
kubectl describe pod my-pod

# Step 3: Check logs
kubectl logs my-pod
kubectl logs my-pod --previous   # if pod restarted

# Step 4: Exec into container if running
```

```
kubectl exec -it my-pod -- /bin/sh
```

Application Failure Troubleshooting

- **Service selector mismatch** — service selector labels do not match pod labels — no traffic reaches pods
- **Namespace mismatch** — service and pod are in different namespaces — DNS name includes namespace
- **Endpoint registration issues** — check endpoints: kubectl get endpoints my-service — if empty, selector is wrong

```
kubectl get endpoints my-service    # check if pods are registered
kubectl get svc my-service -o yaml  # check selector labels
kubectl get pods --show-labels      # check pod labels
```

Control Plane Failure Troubleshooting

```
kubectl get componentstatuses       # check control plane component health
kubectl cluster-info                # cluster endpoint info
kubectl cluster-info dump           # full cluster state dump

# Check static pod logs on master node
kubectl logs -n kube-system kube-apiserver-controlplane
kubectl logs -n kube-system etcd-controlplane

# Check kubelet on master
journalctl -u kubelet -n 50
```

Node Troubleshooting

```
kubectl get nodes                   # check node status
kubectl describe node node1         # detailed node info

# On the node itself
systemctl status kubelet
journalctl -u kubelet -f
```

- **NotReady status** — kubelet is not reporting to apiserver. Check kubelet service on the node.
- **Taint-based evictions** — Node Controller taints node with node.kubernetes.io/not-ready after 40s grace period
- **Grace period** — 40 seconds — Node Controller waits 40s before marking node NotReady
- **Toleration seconds** — 300 seconds default — pods tolerate not-ready taint for 300s before being evicted

xxiv. SYSCTL

What is sysctl

- Interface to read and modify Linux kernel parameters at runtime
- Parameters live in /proc/sys/ — sysctl reads/writes these files
- Used in Kubernetes for networking and performance tuning

Temporary Modification (lost on reboot)

```
sysctl net.ipv4.ip_forward=1        # enable IP forwarding (required for
Kubernetes networking)
sysctl vm.swappiness=0              # disable swap (required for Kubernetes)
sysctl -a                           # show all current kernel parameters
```

Permanent Modification (survives reboot)

```
# Method 1: Edit /etc/sysctl.conf
echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf
echo "vm.swappiness = 0" >> /etc/sysctl.conf

# Method 2: Add file to /etc/sysctl.d/
echo "net.ipv4.ip_forward = 1" > /etc/sysctl.d/99-kubernetes.conf

# Apply changes without reboot
sysctl --system
```

`sysctl --system` → loads all settings from /etc/sysctl.conf and /etc/sysctl.d/*.conf without rebooting

Common Parameters for Kubernetes

```
net.ipv4.ip_forward = 1     # REQUIRED: allows packet forwarding between network
interfaces
vm.swappiness = 0           # REQUIRED: disables swap — Kubernetes requires swap
to be off
net.bridge.bridge-nf-call-iptables = 1  # bridges send traffic through iptables
(for kube-proxy)
```

xxv. KUBERNETES FROM SCRATCH (Conceptual)

Binary Categories

- **Server binaries (control plane)** — kube-apiserver, kube-controller-manager, kube-scheduler, etcd
- **Node binaries (worker node)** — kubelet, kube-proxy
- **Client binaries** — kubectl

⚠ NOTE: etcd is a separate project — it is NOT part of the Kubernetes binaries. It must be installed separately.

Certificate Authority and Component Certificates

- Kubernetes uses TLS for all communication between components
- Certificate Authority (CA) signs all component certificates
- Each component needs its own certificate for mutual TLS authentication

```
Certificates per component:
kube-apiserver     → server cert, etcd client cert, kubelet client cert
etcd               → server cert, peer cert, client cert for apiserver
kube-scheduler     → client cert to talk to apiserver
controller-manager → client cert to talk to apiserver
kubelet            → server cert, client cert to talk to apiserver
kubectl            → client cert (admin cert)
```

kubeconfig Files

- Each component uses a kubeconfig file to authenticate with the apiserver

```
kubeconfig contains:
- cluster: apiserver URL and CA certificate
- user: client certificate and key for authentication
- context: links a user to a cluster
```

systemd Service Management

- On bare-metal/VM clusters, each component runs as a systemd service

```
# Each component has a service file
/etc/systemd/system/kube-apiserver.service
/etc/systemd/system/kube-scheduler.service
/etc/systemd/system/kubelet.service

# Common systemd commands for K8s components
systemctl status kube-apiserver
systemctl restart kubelet
journalctl -u kube-apiserver -f
```

Common Patterns Across All Components

```
4 common patterns every component needs:

1. Certificates      → TLS cert and key for secure communication
2. kubeconfig        → how to connect and authenticate to apiserver
3. Configuration     → component-specific flags and settings
4. systemd           → service definition to start/stop/restart component
```

xxvi. MISCELLANEOUS BUT IMPORTANT

Command and Arguments in Pod spec

- **command** — overrides the Docker ENTRYPOINT of the container image
- **args** — overrides the Docker CMD of the container image

```
containers:
- name: my-container
  image: ubuntu
  command: ["/bin/sh"]          # overrides ENTRYPOINT
  args: ["-c", "echo hello"]    # overrides CMD
```

`command` → equivalent to Docker ENTRYPOINT

`args` → equivalent to Docker CMD

Environment Variables in Containers

```
containers:
- name: my-container
  env:
  - name: MY_VAR
    value: "hello"
  - name: POD_NAME
    valueFrom:
```

```
      fieldRef:
        fieldPath: metadata.name    # inject pod metadata as env var
  - name: DB_PASS
    valueFrom:
      secretKeyRef:
        name: my-secret
        key: password
```

Labels and Selectors

- **Labels** — key-value pairs attached to resources — used for identification and selection
- **Selectors** — used by Services, ReplicaSets, Deployments to find pods with matching labels

```
# Add label
kubectl label pod my-pod env=prod

# Remove label
kubectl label pod my-pod env-

# Select by label
kubectl get pods -l app=myapp
kubectl get pods -l "app in (myapp, webapp)"
kubectl get pods -l app=myapp,env=prod      # multiple labels (AND condition)
```

Annotations

- Key-value pairs like labels but NOT used for selection
- Used for metadata: build version, contact info, tool-specific config

```
metadata:
  annotations:
    build-version: "1.2.3"
    contact: "allen@cognizant.com"
    nginx.ingress.kubernetes.io/rewrite-target: "/"
```

kubectl explain

- Shows the structure and description of any Kubernetes resource or field

```
kubectl explain pod
kubectl explain pod.spec
kubectl explain pod.spec.containers
kubectl explain pod.spec.containers.resources
kubectl explain deployment.spec.strategy
```

⚠ NOTE: kubectl explain is your best friend in exams. Use it to look up YAML structure without internet access.

Pod Affinity and Anti-Affinity

- **Pod Affinity** — schedule this pod NEAR pods with matching labels (same node or same zone)
- **Pod Anti-Affinity** — schedule this pod AWAY FROM pods with matching labels (spread across nodes)

```
spec:
```

```
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: cache
        topologyKey: kubernetes.io/hostname   # same node
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: myapp
          topologyKey: kubernetes.io/hostname  # spread across nodes
```

`topologyKey: kubernetes.io/hostname` → the "topology" unit — same hostname means same node

`topologyKey: topology.kubernetes.io/zone` → zone-level topology — spread across availability zones

Resource Versions and API Groups

```
Common apiVersion values:
v1                          → Pod, Service, ConfigMap, Secret, PV, PVC, Namespace
apps/v1                     → Deployment, ReplicaSet, DaemonSet, StatefulSet
batch/v1                    → Job, CronJob
networking.k8s.io/v1        → Ingress, NetworkPolicy
storage.k8s.io/v1           → StorageClass
policy/v1                   → PodDisruptionBudget
rbac.authorization.k8s.io/v1 → Role, ClusterRole, RoleBinding, ClusterRoleBinding
autoscaling/v2              → HorizontalPodAutoscaler
scheduling.k8s.io/v1        → PriorityClass
```

CRDs — Custom Resource Definitions

- Extend the Kubernetes API with custom resource types
- Example: Prometheus Operator creates PodMonitor, ServiceMonitor CRDs
- Once CRD is installed you can create custom resources just like built-in ones

```
kubectl get crd                    # list all installed CRDs
kubectl get <custom-resource>      # use custom resources like built-in ones
```

⚠ NOTE: CRDs are awareness-level for most interviews. Know what they are and that tools like Helm, Prometheus Operator, and cert-manager use them.

Kubernetes Documentation Navigation

- Official docs: kubernetes.io/docs
- API reference: kubernetes.io/docs/reference/kubernetes-api/
- kubectl explain is faster than docs during exams

```
# Quick reference for exam
kubectl api-resources           # list all resource types with their apiVersion
kubectl api-versions            # list all available API versions
kubectl explain <resource>      # get YAML structure for any resource
```